CA-MAMBA: what's left to do

The AAMAS submission is in and the arXiv version is ready. This note lists what's still missing for the thesis and for the rebuttal, in the order I'd do it. We don't rerun everything: the submitted method and the main tables stay as they are, most of what's missing can be pulled from runs and logs we already have, and new runs go only where a result is weak or missing.

A few ground rules:

- The method's defaults don't change, since that would mean rerunning every experiment. Improvements, like the parity start in Section 2, are reported as variants on the configurations where they matter.
- All runs in one comparison use the same settings. This has already bitten us on MAMuJoCo: the paper lists tuned optimizer settings (`--actor_lr 3e-5 --model_lr 1e-4 --value_lr 1e-4 --grad_clip 10 --grad_clip_policy 5`), but the code doesn't apply them by default.
- Every new run saves its full config and code commit next to its logs, and is seeded properly (Section 6).
- Read results the way the paper does: average each run's episodes over the last 5k interactions before the checkpoint, then report the mean and sample standard deviation over runs.

1 Fix Table 6 with runs we already have

Table 6 (HalfCheetah, supplement) is the weakest point right now, and it's fixable without new runs. The problem is that it mixes settings: only the *Learned* row used the tuned settings, and all the other rows ran with the code defaults. Here are the runs we already have, regrouped by settings (three per cell, 1M interactions; runs within the limit in parentheses):

		$d = 5$		$d = 25$	
Settings	Rule	Reward	Cost	Reward	Cost
Defaults	Learned, $\beta_0 = 10^{-4}$	2649 ± 73	5.2 ± 8.1 (2/3)	2511 ± 377	10.2 ± 4.4 (3/3)
	Parity start, $\beta_0 = 1$	2266 ± 480	4.0 ± 6.1 (2/3)	2605 ± 235	19.1 ± 25.5 (2/3)
	$\beta = 1$ (A / B)	1954 / 1985	0.6 / 2.4 (3/3)	2282 / 1905	2.7 / 1.2 (3/3)
	$\beta = 0$	2683 ± 1402	474 ± 318 (0/3)	2550 ± 1623	400 ± 381 (0/3)
Tuned	Learned, $\beta_0 = 10^{-4}$	1974 ± 506	18.2 ± 15.9 (1/3)	2527 ± 229	16.6 ± 2.0
	second run set			1667 ± 955	27.6 ± 24.1 (1/3)
	$\beta = 1$	1036 ± 933	0.0 ± 0.0 (3/3)	same runs (β does not read d)	
	$\beta = 0$	3049 ± 2288	607 ± 530 (1/3)	same runs	

What this tells us:

(i) On one setting (the defaults), both learned rules earn more than the fixed price at both limits: 14–36% more at $d = 5$ and 10–37% more at $d = 25$. Mean cost stays at or below the limit, except for the near-zero start at $d = 5$ (5.2). So the weak Table 6 seems to come from mixing settings, not from the method. Three runs per cell are too few to be sure, though.

(ii) On HalfCheetah, the tuned settings don't help the priced rules. At $d = 5$ they earn less (1974 against 2649 for the learned price, 1036 against about 1970 for $\beta = 1$), and at $d = 25$ the two tuned run sets (2527 and 1667) bracket the default set (2511). Appendix C justifies the tuned values by the squashed Gaussian policy, but these runs don't back that up on HalfCheetah.

(iii) No learned rule meets $d = 5$ in every run; the fixed price does, at lower reward. The sentence "parity meets the tight limit at equal reward" (abstract, introduction, Section 5.3, conclusion) is wrong on both settings and has to go.

The default-setting runs are in `data/hc_ablation_endpoints.csv` and `data/local_runs_endpoints.csv`, except the Learned runs at $d = 5$, which are only on the cluster. The tuned reruns are in `~/camamba/runs/t{fixed1,fixed0,learned}_*`.

To do:

- Rebuild Table 6 with every row on the code defaults, from the runs above.
- Before combining them, confirm that all these runs used the same code version. The default-setting rows come from four batches of jobs, run on different days:
  - Imagined, PID, $\beta = 1$ (B) and $\beta = 0$: Slurm jobs 22242437–61 and 22333636–54, branch `feat-aamas`;
  - Parity start and $\beta = 1$ (A): jobs 22313221–30, 22317679 and 22319394;
  - Learned at $d = 25$, which are also the model-error runs: jobs 22132722–24;
  - Learned at $d = 5$: jobs 22400771–74.

  If the logs don't record the commit, match each job's start time against the history of the cluster checkout (`git reflog`), then diff any two commits that differ, in the files the runs use. Rerun only a batch whose code differs in a way that affects training.
- Add two seeds per row, so every cell has five runs. A fixed β doesn't read d, so one new set of fixed-β runs serves both limits.
- Keep the main MAMuJoCo table on the tuned settings; switching it would mean rerunning it. Say in the caption and in Appendix C that Table 6 uses the defaults so that all its rows share one setting, and soften Appendix C's justification of the tuned values.
- Replace the "equal reward" sentence with what the rebuilt table shows. If (i) holds up, the new claim is stronger than the old one: the learned price earns more than a fixed price at both limits, because it reads d.

2 Parity start on the configurations that miss their limit

CA-MAMBA meets only 5 of the 12 nonzero SMAC limits at 100k, which is the first thing a reviewer will hold against us. The paper's answer (RQ2) is that the learned price is too slow: at $d = 0$ on `8m` it only reaches 0.025–0.54 by 100k, and a price fixed at 1 beats it on both cost and win rate (1.75 and 0.97 against 2.05 and 0.95). Starting the price at parity ($\beta_0 = 1$) is the direct fix that diagnosis suggests. Both advantages are standardized, so $\beta = 1$ weighs cost and reward equally. From there the update lowers β when the limit is loose (on HalfCheetah it ends at 0.32–0.51 at $d = 25$) and keeps it near 1 when the limit binds (0.87–1.01 at $d = 5$).

To do:

- Run the parity start on `8m` at $d = 0$ (a new row in Table 2) and on the 7 SMAC configurations where CA-MAMBA misses its nonzero limit, three seeds each.
- If it meets more of those limits without lowering the win rate, report it in the RQ2 section as the fix the diagnosis predicts. That turns "the method rarely meets its limits" into "the method

misses limits for a known reason, and here is the fix". It stays a variant, not the new default, so nothing else needs rerunning.

3 A real "MAMBA + SafeDreamer" baseline

The main AAAI complaint was that the method looks like MAMBA plus SafeDreamer. The paper now explains the difference: when SafeDreamer trains its policy in imagination (BSRP-Lag), its multiplier follows the imagined cost return with an augmented-Lagrangian step, and Table 2's imagined row uses that signal with a plain step. That's an argument, not an experiment, and it rests on three runs where the cost difference is within one standard deviation.

To do:

- Implement SafeDreamer's update exactly (imagined cost return, augmented Lagrangian, its threshold convention) inside MAMBA, and run it on `8m` at $d = 0$, one more SMAC configuration, and HalfCheetah at $d = 5$ (code defaults, to match the rebuilt Table 6), with five seeds.
- Add two seeds to Table 2's imagined and measured rows, so the comparison on `8m` is five against five.

4 More seeds only where results are fragile

Everything else stays at three seeds. The SMAC claim rests on per-map sign tests over 12 maps, which already account for run-to-run noise, and the rebuttal can say so.

To do:

- `bane_vs_bane`: CA-MAMBA's three runs end at win rates 0.19, 0.68 and 0.97. Add two seeds at each of its two limits.
- Table 6 and Table 2: covered in Sections 1 and 3.
- `8m` at $d = 4$: two run sets with the same code and settings disagree (cost 3.98 ± 0.68 against 5.03 ± 0.42). Pool the six runs instead of rerunning.

5 Analysis on data we already have

These need no new training, only collecting the logs of the reported runs (Slurm output files, wandb). If a run's logs are lost, rerun a few representative maps rather than everything.

- *Run manifest.* For every reported number: configuration → run ids → settings → commit. The table generators should read only runs listed there.
- *Cost paid during learning.* CA-MAMBA logs every episode's cost, so from the existing logs report the cumulative excess $E_K = \sum_{e \le K} [C_e - d]_+$ (the sum, over the first K episodes, of each episode cost C_e above the limit d) and the number of violating episodes V_K at each checkpoint, on SMAC and MAMuJoCo. Do the same for the baselines from their training logs. This answers "is learning safer, or only the final policy?"
- *Compute.* Elapsed time and peak memory of the existing jobs from Slurm accounting (`sacct`), or from log timestamps if the jobs are too old. Report each method's time to reach CA-MAMBA's 100k performance.
- *Figures.* The new SMAC learning curves (Appendix E) are ready (`figures/smac_curves.pdf`). Redraw the β trajectories (Appendix J) from the reported runs' logs; the current ones come from older runs.

- *Feasibility against budget.* A small table of how many of the 12 nonzero SMAC limits each method meets at 100k, 1M and 5M.
- *Cost-head recall on SMAC.* How well does the cost head catch cost events on `8m` and `1c3s5z`? Report precision, recall and the area under the precision–recall curve on held-out replay, plus the error of the 14-step predicted cost. Use saved checkpoints and replay if we have them; otherwise one new run per map with the extra logging. Right now our model-error evidence is average-case only (Limitation 2).
- *Model error against violations.* Using the 18 runs we already measured, check whether a larger under-prediction goes with more cost above the limit. That would turn Theorem 4.1 from a decomposition into a measured explanation. The fifteen HalfCheetah runs used the defaults, so they already match the rebuilt Table 6.

6 Small fixes and checks

- Seed every library and every worker from `--seed`, for all new runs. Right now the flag only labels the run; it doesn't seed Python, NumPy, PyTorch, the environments or the Ray workers. The existing runs stay documented as unseeded (Appendix C already says so).
- Write down MAPPO-Lagrangian's SMAC settings (they aren't anywhere), and check the MACPO settings table against the launch commands (total steps, rollout length, safety discount).
- Compare our MAMuJoCo baseline runs with the numbers published with SafePO, so we can say the baselines are reasonable. At 100k MACPO wins no episode on 22 of 24 SMAC configurations, and reviewers may read that as an under-tuned baseline.
- MAPPO-Lagrangian's logged episode cost sometimes exceeds the number of allied units (10.11 on `MMM`, which has ten units). Find out why and fix the accounting if it differs from ours.
- Find out which settings the $\alpha_\beta = 10^{-4}$ runs in the tight-limit table used (the paper's "tenfold step" result).
- `scripts/generate_mamujoco_tables.py` rounds an already rounded value, so HalfCheetah at $d = 5$ shows 1973 in one table and 1974 in another (the mean is 1973.48). Round from the per-seed values.

7 Writing the thesis

- *Background:* constrained Dec-POMDPs; recurrent state-space models (prior, posterior, KL balancing); Dreamer and MAMBA; PPO, generalized advantage estimation and λ-returns; Lagrangian relaxation, dual updates and PID control; SMAC and MAMuJoCo.
- *Related work:* constrained MARL, model-based MARL, safe RL with world models (SafeDreamer, LAMBDA) and shielding, each with what it does and how CA-MAMBA differs.
- *Method:* the full architecture, losses and training loop (Appendices A–C), with the reason for each design choice, and the parity start as a variant.
- *Theory:* grow the chapter around the existing proofs (Appendix L): intuition for each term, the bootstrap corollary, the team term η of Proposition 4.2, and the counterexample.
- *Results:* every configuration and learning curve, all ablations, and the negative results too (missed limits, the PID collapse, run-to-run variance). Every number should come out of a script that has a test.
- *Reproducibility:* code, configs, run manifest and scripts, ready to release once the paper is accepted.

8 If there's time

- *Communication trained end to end.* Today we only remove messages at acting time, from a model trained with them: reward doesn't drop on Ant 4×2, drops 19% on Ant 2×4 (within the spread), and falls from 2527 to 982 on HalfCheetah (standard deviation 1303). A self-only model trained end to end on HalfCheetah and Ant 4×2, with the per-agent transition errors of Proposition 4.2 measured with and without messages, would be the real answer to "what do messages add?"

9 What we're dropping

- *Rerunning every experiment* with five seeds or with a changed method: too expensive, and Sections 1–5 cover what it would have answered.
- *Switching MAMuJoCo to the default settings everywhere:* it would mean rerunning the main MAMuJoCo table.
- *Unconstrained MAMBA as a baseline:* the cost-blind $\beta = 0$ row already is that baseline.
- *Rerunning the safety ablation (Appendix G):* Table 2's cost-blind row replaces it, so drop the old figure.
- *Dense collision cost (Appendix F):* its table was copied by hand from an older code branch; cut it from the supplement rather than rebuild it.
- *Per-agent constraints:* both benchmarks charge every agent the same team cost, and testing per-agent constraints needs a new task. We state it as a limitation and future work.
- *More MAMuJoCo tasks and SMAC beyond 100k:* expensive, and they don't answer a known objection.
- *A separate evaluation pass, a MAPPO-Lagrangian hyperparameter sweep, an adaptive multiplier step, the SMAC communication ablation, and Table 2 on more maps:* nice to have, but low priority. The parity-start runs of Section 2 already put the multiplier on several more SMAC configurations.